

Exercises on recursive functions

Implement the following recursive functions together with a main testing their behaviour.

1. m-th summation of first n natural numbers.

Write a function that calculates the `m`-th summation of first `n` natural numbers, which is defined as

```
If m > 1
    SUM(n, m) = SUM(SUM(n, m - 1), 1)
Else
    SUM(n, 1) = Sum of first n natural numbers.
```

The sum of first `n` natural numbers corresponds to the formula

$$n * (n + 1) / 2$$

2. gcd

Write a function that calculates the greatest common divisor using the Euclidean algorithm. Given two positive integers `a` and `b` (assumed non both null):

```
gcd(a, b) = gcd(b, a mod b) if b > 0
gcd(a, 0) = a
```

3. print a pattern (1)

Write a function that, given an integer `n`, prints the following pattern:

```
Input: n = 16
Output: 16, 11, 6, 1, -4, 1, 6, 11, 16

Input: n = 10
Output: 10, 5, 0, 5, 10
```

and so forth

4. print a pattern (2)

Write a function that, given a positive integer `n`, prints the following pattern:

```
Input: n = 0
Output:

Input: n = 1
Output:
-

Input: n = 2
Output:
-
--
-

Input: n = 3
Output:
-
--
-
---
-
--
-
```

and so forth

5. binary numbers

Write a function that checks whether a positive integer `n` is made up of only zeroes and ones (for example, `0`, `1`, `101`, `1101`). Remember that `n mod 10` results in the rightmost digit while `n / 10` cuts off the rightmost digit.

6. binary search

Define the following function that implements the binary search algorithm recursively using the array name as a pointer to the first element of the array

```
/* given an array, which is assumed to contain values in increasing order, searches for the specified value
returns a pointer to the corresponding element in the array if the search succeeds; the NULL pointer otherwise */

int * search(int val, int a[], int a_size)
```